Databases

Matt ter Steege
matttersteege@gmail.com
Universiteit Utrecht
Utrecht, Nederland

Contents

Contents 1
1 Relationele model & algebra 2
1.1 defenities 2
1.2 Keys 2
1.3 Selectie & Projectie 2
2 Enity Relationship diagrams (ERD) 3
3 Functional dependencies & normalisatie 3
3.1 Normalisatietheorie 3
3.2 Functional dependencies 3
3.3 Decompositie 4

1 Relationele model & algebra

1.1 defenities

Relatieschema: Movie			
movid	**title**	**year**	**rating**
tt0469494	There will be blood	2007	8,1
tt0086879	Amadeus	1984	8,4
tt0102926	The silence of the lambs	1991	8,6
...	...	...	...

Uitleg gekoppeld aan de tabel:

- **Movie** is het **relatieschema** R R(movid, title, year, rating)
- De **kolomnamen** zijn de **attributen** $attr(R) = \{$movid, title, year, rating$\}$
- Het aantal attributen is 4, dus de **degree** is 4
- Elke rij in de tabel is één **tuple** Bijvoorbeeld: < tt0469494, There will be blood, 2007, 8,1 >
- Elk attribuut heeft zijn **domein**:
  - movid → string
  - title → string
  - year → integer
  - rating → float
- Alle rijen samen vormen de **relatie** $r(Movie)$

1.2 Keys

- **Key** is een attribuut of combinatie van attributen die een tuple uniek identificeert binnen een relatie. Bijvoorbeeld: movid is een key voor Movie.
- **Superkey** is een attribuut of combinatie van attributen die een tuple uniek identificeert, maar kan ook extra attributen bevatten. Bijvoorbeeld: (movid, title) is een superkey voor Movie.
  Definitie: Een set van attributen $K \subseteq attr(R)$ is een superkey als voor elke relatie $r(R)$: $\forall t_1, t_2 \in r(R) : t_1[K] = t_2[K] \Rightarrow t_1 = t_2$. of: "Als twee tuples dezelfde waarden hebben voor alle attributen in K, dan zijn het dezelfde tuples."
- **Candidate key** is een minimale superkey, wat betekent dat het geen extra attributen bevat die niet nodig zijn voor unieke identificatie. Bijvoorbeeld: movid is een candidate key voor Movie. Definitie: Een set van attributen $K \subseteq attr(R)$ is een candidate key als K een superkey is en geen echte subset van K een superkey is. of: "Er is geen kleinere subset van attributen die ook een superkey is."
- **Primary key** is de gekozen candidate key die wordt gebruikt om tuples uniek te identificeren binnen een relatie. Bijvoorbeeld: movid is de primary key voor Movie.
- **Foreign key** is een attribuut of combinatie van attributen in een relatie die verwijst naar de primary key van een andere relatie. Bijvoorbeeld: in een relatie Actor met attribuut movid als foreign key die verwijst naar de primary key movid in Movie. Dit kan je gebruiken om aan te geven in welke films een acteur heeft gespeeld. Definitie: Een set attributen $K \in attr(R)$ is een foreign key van relatie R_j als candidate key. $R_i[K]$ verwijst naar de primary key van relatie R_j. of: "De waarden van K in R_i moeten overeenkomen met de waarden van de primary key in R_j."
- **Key constraints** zorgen ervoor dat de waarden van de key-attributen uniek zijn binnen de relatie en niet null mogen zijn.
- De **referential integrity constraints** zorgen ervoor dat de waarden van de foreign key-attributen overeenkomen met de waarden van de primary key in de gerelateerde relatie, of null zijn als er geen overeenkomst is.

1.3 Selectie & Projectie

- **Selectie** (σ_p), σ is een operator die rijen (tuples) selecteert uit een relatie op basis van het predicaat p.
- **and** ($\wedge$) en **or** ($\vee$) worden gebruikt om meerdere voorwaarden te combineren in het predicaat. Voorbeeld: $\sigma_{\text{year}>2000 \wedge \text{rating}>8}(Movie)$ selecteert alle films uit de Movie-relatie die zijn uitgebracht na het jaar 2000 en een beoordeling hoger dan 8 hebben.
- **Projectie** (π_L), π is een operator die kolommen (attributen) selecteert uit een relatie op basis van de lijst van attributen L. Voorbeeld: $\pi_{\text{title,year}}(Movie)$ projecteert alleen de titel en het jaar van de films uit de Movie-relatie.
- **union** ($\cup$), de unie-operator combineert de resultaten van twee relaties door alle unieke tuples uit beide relaties te behouden. Voorbeeld: $R_1 \cup R_2$ geeft alle unieke tuples die voorkomen in zowel relatie R_1 als relatie R_2.
- **intersection** ($\cap$), de intersectie-operator retourneert de tuples die voorkomen in beide relaties. Voorbeeld: $R_1 \cap R_2$ geeft alle tuples die in zowel relatie R_1 als relatie R_2 voorkomen.
- **set difference** ($-$), de verschil-operator retourneert de tuples die voorkomen in de eerste relatie maar niet in de tweede. Voorbeeld: $R_1 - R_2$ geeft alle tuples die in relatie R_1 voorkomen maar niet in relatie R_2.

- **cartesian product** ($\times$), het cartesisch product combineert elke tuple van de eerste relatie met elke tuple van de tweede relatie. Voorbeeld: $R_1 \times R_2$ geeft alle mogelijke combinaties van tuples uit relatie R_1 en relatie R_2.
- **thetha-join** ($\bowtie_\theta$), de theta-join-operator combineert tuples uit twee relaties op basis van een opgegeven voorwaarde θ. Voorbeeld: $R_1 \bowtie_{\text{R1.attr=R2.attr}} R_2$ combineert tuples uit relatie R_1 en relatie R_2 waar de waarden van het attribuut attr gelijk zijn.
- **natural join** ($\bowtie$), de natural join-operator combineert tuples uit twee relaties op basis van gemeenschappelijke attributen met dezelfde naam. Voorbeeld: $R_1 \bowtie R_2$ combineert tuples uit relatie R_1 en relatie R_2 waar de waarden van de gemeenschappelijke attributen gelijk zijn.
- **division** ($\div$), de division-operator retourneert de tuples uit de eerste relatie die overeenkomen met alle tuples in de tweede relatie. Voorbeeld: $R_1 \div R_2$ geeft alle tuples uit relatie R_1 die overeenkomen met alle tuples in relatie R_2 op basis van een gemeenschappelijk attribuut.
- **Assignment & renaming** $T :=<\text{alg}_e xpr >, deassignment-operatorwijsthetresultaatvaneenrelationelealgebra-expressietoeaaneennieuwerelatie\text{T.V} := \sigma_{\text{year}>2000}(Movie)$ creëert een nieuwe relatie T die alle films bevat die zijn uitgebracht na het jaar 2000.
- **Renaming on the fly** $(\rho(T)(<\text{alg}_e xpr >), derenaming-operatorwijsthetresultaatvaneenrelationelealgebra-expressietoeaaneennieuwerelatie\text{Tzond}$ creëert een tijdelijke relatie T die alle films bevat die zijn uitgebracht na het jaar 2000, maar deze relatie is alleen beschikbaar binnen de context van deze expressie.

Voorbeeld **(bibliotheek schema)**:

Stel je hebt de volgende schema's voor een bibliotheekdatabase:

- **Book**(book_id, title, author, year)
- **Member**(member_id, name, join_date)
- **Loan**(loan_id, book_id, member_id, loan_date, return_date)

Geef alle namen van lezers die minstens één boek hebben geleend van Dickens.

$$\pi_{\text{name}}(\text{Reader} \bowtie (\text{Loan} \bowtie \sigma_{\text{author}='Dickens'}(\text{Book})))$$

Uitleg:

- $\sigma_{\text{author}='Dickens'}(\text{Book})$ selecteert alle boeken geschreven door Dickens.
- $\text{Loan} \bowtie \sigma_{\text{author}='Dickens'}(\text{Book})$ combineert de leningen met de geselecteerde boeken om alle leningen van Dickens' boeken te vinden.
- $\text{Reader} \bowtie (\text{Loan} \bowtie \sigma_{\text{author}='Dickens'}(\text{Book}))$ combineert de lezers met de leningen van Dickens' boeken om alle lezers te vinden die deze boeken hebben geleend.
- $\pi_{\text{name}}(\ldots)$ projecteert alleen de namen van deze lezers.

---

2 Enity Relationship diagrams (ERD)

Als je een database wilt maken, dan is het belangrijk dat je het domein goed modelleert. Een ERD is een grafische weergave van de entiteiten, attributen en relaties binnen een domein. Het helpt bij het ontwerpen van databases door de structuur en de onderlinge verbanden duidelijk te maken.

ER diagrammen hebben verschillende componenten:

- **Rechthoek** Geeft een entiteit weer, bijvoorbeeld een persoon, plaats of ding.
- **Ovaal** Geeft een attribuut weer, bijvoorbeeld een naam, leeftijd of adres.
- **Ruit** Geeft een relatie weer tussen entiteiten, bijvoorbeeld "werkt bij" of "heeft".
- **Lijn** Verbindt entiteiten, attributen en relaties om de structuur van het domein te tonen.

Deze lijnen/relaties kunnen ook kardinaliteiten hebben, zoals 1:1, 1:N of M:N, om aan te geven hoeveel entiteiten er betrokken zijn bij de relatie.
1:1 (één-op-één) betekent dat één entiteit van het ene type overeenkomt met precies één entiteit van het andere type.
1:N (één-op-veel) betekent dat één entiteit van het ene type overeenkomt met meerdere entiteiten van het andere type.
M:N (veel-op-veel) betekent dat meerdere entiteiten van het ene type overeenkomen met meerdere entiteiten van het andere type.

Ook kan je deelname aan een relatie aangeven als verplicht (1,...) of optioneel (0,...), wat aangeeft of een entiteit deel moet nemen aan de relatie of niet.

Zwakke entiteiten kunnen alleen bestaan in relatie tot een andere entiteit, en hebben een gedeeltelijke sleutel die samen met de sleutel van de gerelateerde entiteit een unieke identificatie vormt. Bijvoorbeeld een afboeking als je iets koopt kan alleen als je een bankrekening hebt, en de afboeking heeft een gedeeltelijke sleutel die samen met de rekeningnummer van de bankrekening een unieke identificatie vormt.

3 Functional dependencies & normalisatie

3.1 Normalisatietheorie

Wanneer is een databaseontwerp goed?

3.2 Functional dependencies

Stel je hebt een universum $R(A, B, C, D)$. Je hebt de regels $A \to B, B \to C, A \to C$

Dan kan je concluderen dat $A \to AB \to ABC$, ofwel $A \to ABC$.

Stel je hebt $A \to BCD$, dan kan je zeggen $A \to B$

Voorbeeld **(Voorbeeld van functionele afhankelijkheden)**:

We hebben een schema $R(ABCDEFG)$ en een set FD's: $F = C \to DE, A \to C, G \to D, B \to G$
We willen de closure van A berekenen, oftewel A^+, om te zien welke attributen functioneel afhankelijk zijn van A.
We beginnen met $A^+ = \{A\}$ (omdat A altijd functioneel afhankelijk is van zichzelf).
We kijken naar de FD's in F en zien dat $A \to C$ in F zit, dus we voegen C toe aan A^+: $A^+ = \{A, C\}$.
Nu kijken we opnieuw naar de FD's en zien dat $C \to DE$ in F zit, en aangezien C al in A^+ zit, kunnen we D en E ook toevoegen aan A^+: $A^+ = \{A, C, D, E\}$.
We kijken nogmaals naar de FD's, maar er zijn geen FD's meer die we kunnen toepassen op de attributen in A^+, dus we zijn klaar.
Stel wel willen $(AB)^+$ weten (notitie, dat is dus NIET AB^+, want dat is $A(B)^+$)

3.3 Decompositie

Voorbeeld **(XYZ-regel (Verliesvrij))**:

Gegeven $R(ABCDE)$ en FD's $F = \{A \to C,\ C \to BE,\ A \to E\}$.
Beschouw de decompositie $R_1 = (AC)$ en $R_2 = (ABDE)$.

De gemeenschappelijke attributen zijn $X = R_1 \cap R_2 = \{A\}$.
Schrijf $Y = R_1 \setminus X = \{C\}$ en $Z = R_2 \setminus X = \{B, D, E\}$.

Volgens de XYZ-regel is de decompositie verliesvrij als $X \to Y$ of $X \to Z$ geldt (met betrekking tot F^+).
Controleer $A \to C$ en $A \to BDE$:
uit F volgt $A \to C$ en $A \to E$, maar er is geen $A \to B$ en geen $A \to D$.
Dus geldt $A \to C$ maar niet $A \to BDE$.

Conclusie: de decompositie (AC) $(ABDE)$ is verliesvrij volgens de gegeven FD's.

Voorbeeld **(XYZ-regel (niet verliesvrij))**:

Gegeven $R(ABCDE)$ en FD's $F = \{A \to C,\ C \to BE,\ A \to E\}$.
Beschouw de decompositie $R_1 = (AC)$ en $R_2 = (CBDE)$.

De gemeenschappelijke attributen zijn $X = R_1 \cap R_2 = \{C\}$.
Schrijf $Y = R_1 \setminus X = \{A\}$ en $Z = R_2 \setminus X = \{B, D, E\}$.

Volgens de XYZ-regel is de decompositie verliesvrij als $X \to Y$ of $X \to Z$ geldt (met betrekking tot F^+).
Controleer $C \to A$ en $C \to BDE$:
uit F volgt $C \to B$ en $C \to E$, maar er is geen $C \to A$ en geen $C \to D$.
Dus geldt noch $C \to A$ noch $C \to BDE$.

Conclusie: de decompositie (AC) $(CBDE)$ is niet verliesvrij volgens de gegeven FD's.